

Beyond the GPU

Why AI is driving a second hardware transition — a systems view for software engineers

Condensed walkthrough • ~1 hour

TALKING POINTS

1 / 23

Framing. Two hardware transitions: CPU→GPU, and now GPU→specialized accelerators. Audience is software/systems, so I introduce each hardware concept conceptually. Core thesis: software optimization keeps hitting PHYSICAL limits — the limits, not software failure, drive specialization. Set expectation: light on products, heavy on the physics and economics.

Where we are going

1

The driving factors

The physical pressures that push work off the GPU: data movement, memory bandwidth, energy, determinism.

2

The decision framework

Six architectural contracts and a four-condition test for when specialization is actually justified.

3

The families

How beyond-GPU accelerators group by which contract they change — not by vendor.

4

The landscape, grouped

Incumbents, hyperscaler silicon, and challengers — read through the framework, not as a catalog.

5

Economics

Why lifetime cost and software capital decide most outcomes.

6

Conclusions

What holds across the whole compute continuum.

⇒ Less time on products; more on the physics and economics that make the choice decidable.

Beyond the GPU — 1-hour walkthrough

2

TALKING POINTS

2 / 23

Roadmap. Deliberately NOT a product tour — the products change every quarter; the framework doesn't. Six movements. Tell them the payoff up front: by the end they'll have a repeatable test for "is a specialized accelerator justified here?"

Six hardware ideas, in software terms



GPU

Thousands of parallel lanes. Fast when work is regular, batched, and streamed from high-bandwidth memory.



Matrix / tensor engine

A block that does a whole matrix tile per instruction — amortizes control over many multiply-adds.



HBM

Stacked DRAM giving multi-TB/s. The scarce resource: capacity and bandwidth, not arithmetic.



NPU

An SoC block plus a compiler that maps a whole model graph onto it at low energy.



TOPS

A peak arithmetic rating. Rarely the bottleneck — treat as marketing, not a benchmark.



Contract

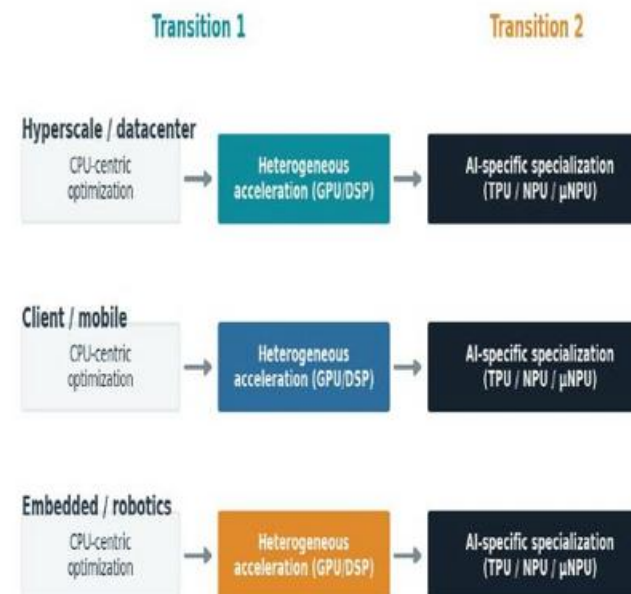
What hardware promises software about execution, memory, numbers, scheduling, comms, and tooling.

TALKING POINTS

3 / 23

Vocabulary for a software audience — spend ~2 min. The two ideas to plant: (1) HBM bandwidth, not FLOPS, is usually the scarce resource; (2) "contract" — hardware makes promises to software, and specialization means CHANGING one of those promises. TOPS is marketing; return to this when we hit the client landscape.

Two transitions, across the whole continuum



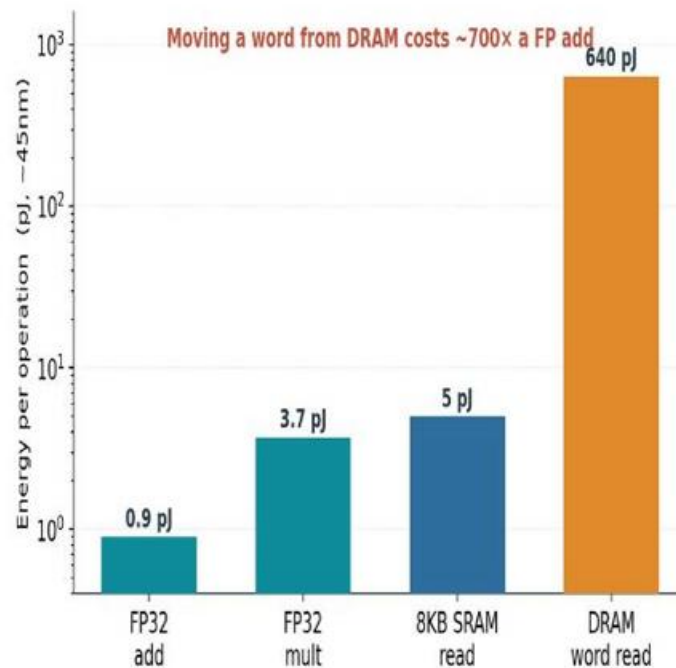
⇒ Software optimization keeps hitting limits; each transition adds specialization, it does not replace what came before.

TALKING POINTS

4 / 23

The organizing thesis. Both transitions occurred because software optimization exposed limits it could no longer move economically — not because software failed. Stress: each transition ADDS an execution domain; CPUs and GPUs don't go away. The three lanes (datacenter / mobile / embedded) hit the SAME pattern under DIFFERENT dominant constraints — we'll follow all three.

Data movement — not arithmetic — dominates energy



- Compute is nearly free.
- Moving operands is not.
- So locality and reuse set efficiency, not FLOPS.

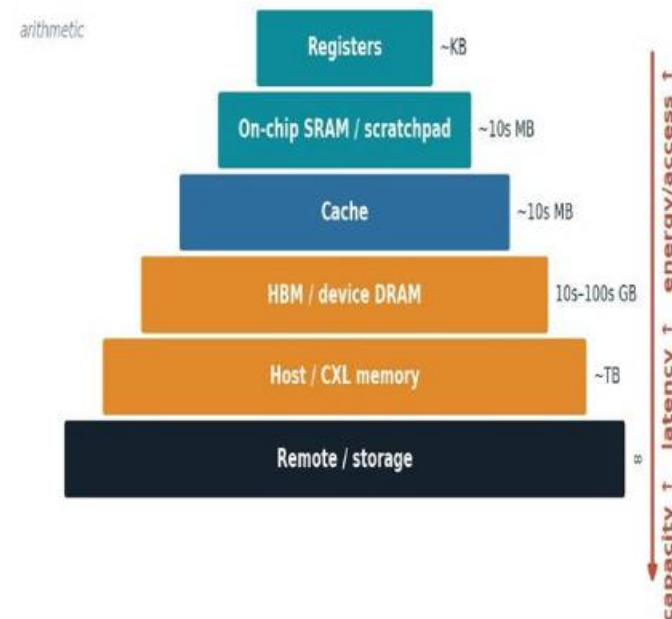
⇒ Every architecture that follows is a different answer to 'move less data, or move it a shorter distance.'

TALKING POINTS

5 / 23

The single most important hardware fact for software people. Horowitz 45nm numbers: an FP add ~0.9 pJ; a DRAM word read ~640 pJ — roughly 700x. So the bottleneck and the energy bill are set by data movement, not by the math unit. This one slide justifies almost every family later: they all move less data or move it a shorter distance. (Numbers are process-node specific but the ratio holds.)

Memory is a distance hierarchy



- Each step out: more capacity.
- Also more latency and energy.
- Caches help only if data is reused before eviction.

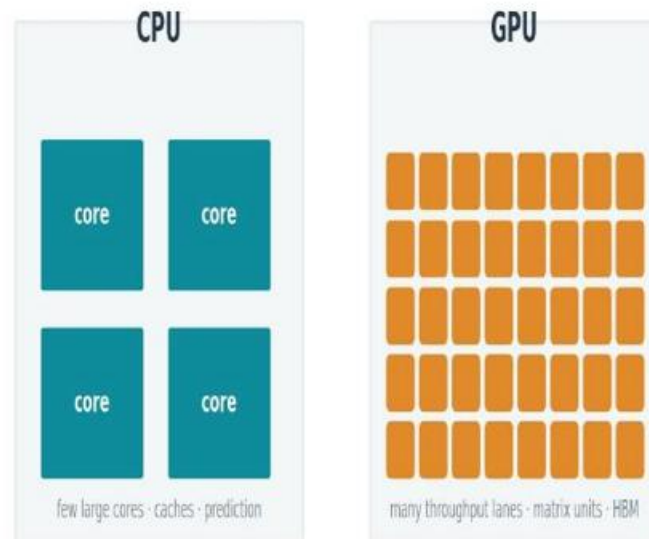
⇒ 'Memory-bound' is many different problems — each one a lever a specialized design can pull.

TALKING POINTS

6 / 23

Memory is not one thing — it's a ladder trading capacity for latency and energy. "Memory-bound" hides many distinct problems: bandwidth, capacity, cache-line overfetch, irregular access, too little reuse. Caches only pay off when the working set is reused before eviction. Each rung is a different lever a specialized design can pull — remember this at the families section.

CPU and GPU spend silicon differently



- CPU: low-latency across diverse code.
- GPU: throughput across regular work.
- Neither is optimal for every phase.

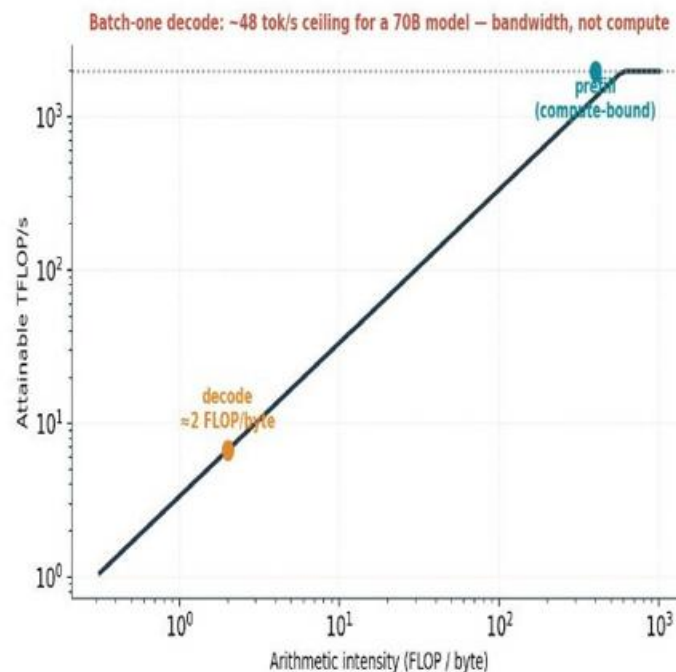
⇒ Specialization narrows what a design does in exchange for more useful work per unit area and energy.

TALKING POINTS

7 / 23

Where the transistors go. CPU spends area on making diverse, branchy code fast at low latency (caches, prediction, out-of-order). GPU spends it on many throughput lanes. Specialization is just the next step on the same axis: narrow the job further, get more useful work per mm^2 and per joule — but only for work that fits.

Batch-one decode is a bandwidth wall



- 70B model, 8-bit, one sequence.
- ≈48 tokens/s ceiling on an H100.
- ~0.3% of the matrix throughput usable.

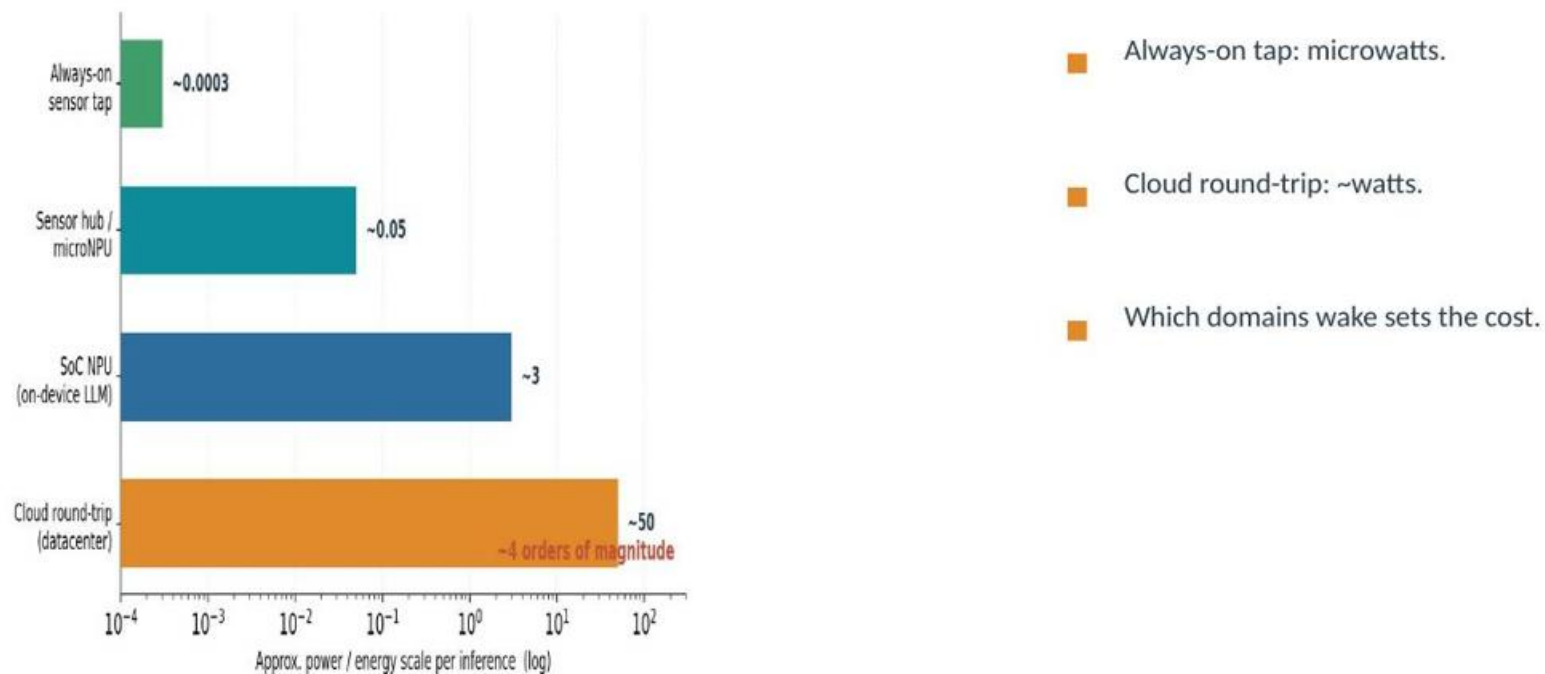
⇒ No amount of extra arithmetic moves this — it is set by memory bandwidth. This is the pressure that reshapes serving silicon.

TALKING POINTS

8 / 23

The keystone worked example. Generating one token for one sequence means streaming ALL ~70 GB of weights through HBM. At ~3.35 TB/s that's ~48 tokens/s — and you use ~0.3% of the matrix throughput. Author-computed from published datasheets; idealized upper bound. Punchline: adding FLOPS does nothing; only batching (amortize weight reads across sequences) or a changed memory contract helps. This is WHY 2025-26 silicon is going "inference-first" and memory-first.

Energy per inference spans four orders of magnitude



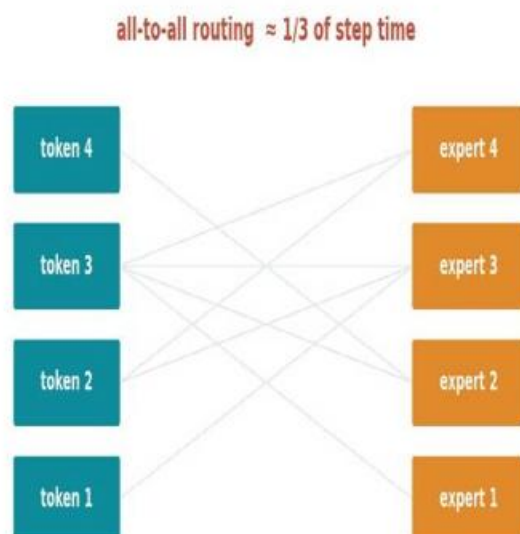
⇒ On-device always-on features are impossible on a GPU contract — the wake-scope bound is structural, not an efficiency gap.

TALKING POINTS

9 / 23

"Wake scope" = which power domains must be awake to answer. It spans ~ 4 orders of magnitude, microwatts (always-on keyword spotting) to watts (cloud round-trip). Key: an always-on feature isn't just cheaper on a tiny engine — it's IMPOSSIBLE within a phone's battery budget on a GPU's submission/memory contract. This is a structural bound, and it's the strongest beyond-GPU territory (comes back at the NPU slide).

Mixture-of-experts turns compute into communication



⇒ Routing and all-to-all can be $\sim 1/3$ of step time — a bound in the fabric, not the math unit.

TALKING POINTS

10 / 23

Mixture-of-experts routes each token to a few experts, which may live on different devices. The all-to-all exchange can be $\sim 1/3$ of a training step. So as models get sparser, the bottleneck moves into the NETWORK FABRIC, not the arithmetic — which is why interconnect (NVLink, UALink, Ethernet) becomes a competitive axis later.

Software succeeds — until it meets physics

3×

IO-aware attention
(FlashAttention) throughput

2–4×

KV paging / continuous
batching (vLLM)

~34%

of a training step spent in MoE
all-to-all

<300 μ W

always-on keyword spotting
budget

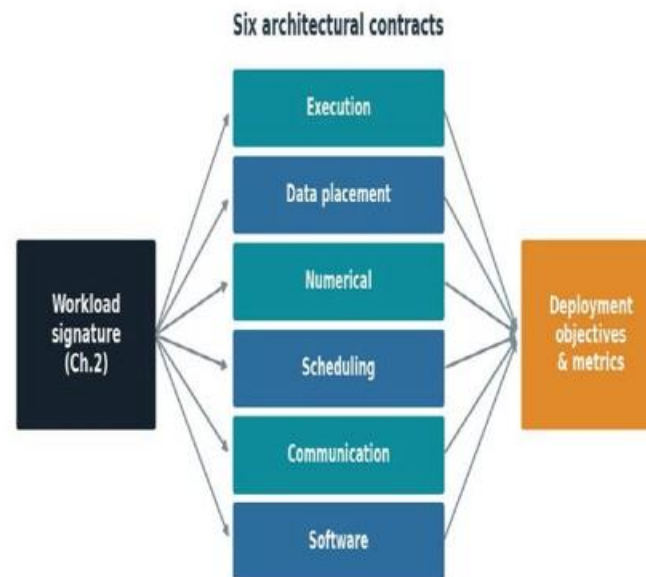
⇒ Every gain is real; every gain eventually exposes a residual bound software cannot move. That residue is the case for hardware.

TALKING POINTS

11 / 23

Be fair to software first — this earns credibility. FlashAttention ~3× from IO-awareness, vLLM 2-4× from paged KV + continuous batching. These are huge and real. BUT each success reveals a residual bound it can't cross (MoE all-to-all still ~34%; always-on still <300 μ W). The hardware case is exactly that RESIDUE — never "software failed."

Hardware defines six contracts



⇒ A workload signature stresses some contracts, not others — that mismatch is where a new architecture earns its place.

TALKING POINTS

12 / 23

Framework, part 1. Any processor makes six promises to software: execution, data placement, numerical, scheduling, communication, software/tooling. A workload's "signature" stresses only SOME of them. A new architecture is justified only where it changes a contract the workload actually stresses — otherwise you've built a faster answer to a question nobody asked.

Is specialization justified? A four-condition test



- 1 · residual bound remains
- 2 · it has a material consequence
- 3 · hardware changes the bound
- 4 · lifetime economics repay it

⇒ Same architecture, opposite verdicts. Condition 4 — volume and software cost — decides most cases.

TALKING POINTS

13 / 23

Framework, part 2 — the test that makes the whole talk decidable. Four conditions, ALL must hold. Three real outcomes from identical reasoning: TPU v1 → specialize (all four held, published 15-30× throughput, 30-80× perf/W under latency SLOs); Nervana/Graphcore → good silicon, condition 4 failed (no volume, CUDA switching cost); enterprise private serving → stay on GPU (conditions 2 & 4 fail). Condition 4 = volume + software enablement cost — it decides the majority of real cases.

What the GPU absorbs — and what it cannot

Figure 5.5 — Partition of residual bounds by absorbability
Residual bounds from Chapter 2, partitioned by whether GPU evolution can absorb them without abandoning the platform contract (host-driven submission, SMT programmability, shared software capital)

ABSORBED INTO THE GPU PLATFORM (evidence class A)	CONTESTED MIDDLE (decided by §2.14 condition 4)	STRUCTURALLY OUTSIDE THE THROUGHPUT CONTRACT
Matrix density → tensor cores	Low-batch decode bandwidth bound	Wake-scope / always-on energy (AI vs ML domains)
Precision → FP16/BF16/FP8 engines	MoE routing / all-to-all (~1/3 step time)	Provable worst-case timing and certification
Structured sparsity → 2-4 paths	Near-memory embedding lookup	Near-sensor locality
Movement overhead → async copy / TMA	Phase-disaggregated serving	Embedded per-unit cost floors
Launch overhead → graph capture		
Scale-up comms → NVLink domains	CPU trajectory + software capital vs. contract-changing designs	Secure basis of the beyond-GPU case
Isolation → device partitioning		

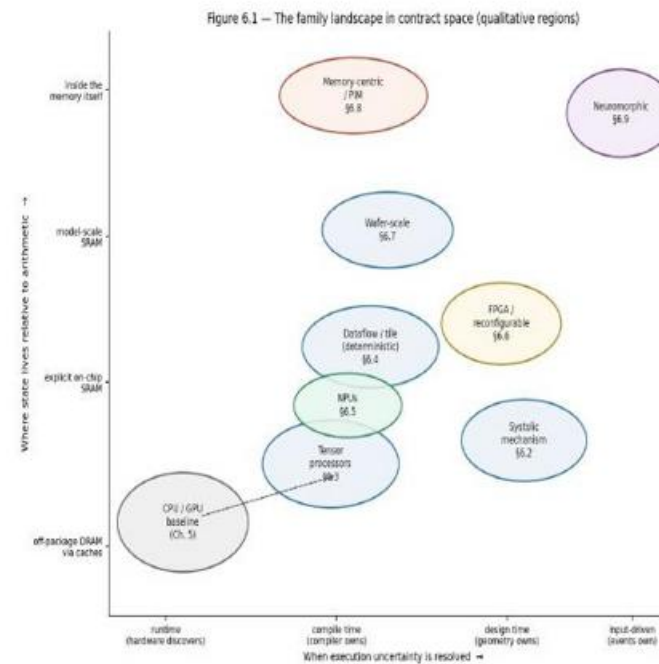
⇒ Absorbable bounds go to the platform; structural bounds (wake scope, determinism, near-sensor, cost floors) belong to specialists.

TALKING POINTS

14 / 23

Why the GPU keeps winning the middle — and where it can't. **ABSORBED** into the GPU platform: matrix density, narrow precision (FP8/FP4), movement overhead, scale-up comms — these just become GPU features (tensor cores, NVLink). **STRUCTURALLY NOT** absorbable: wake-scope energy, provable worst-case timing, near-sensor locality, embedded cost floors — absorbing them would mean abandoning what makes a GPU a GPU. The **CONTESTED** middle (low-batch decode, MoE routing, near-memory lookup) is exactly where the commercial fights happen.

One map: when uncertainty resolves × where state lives



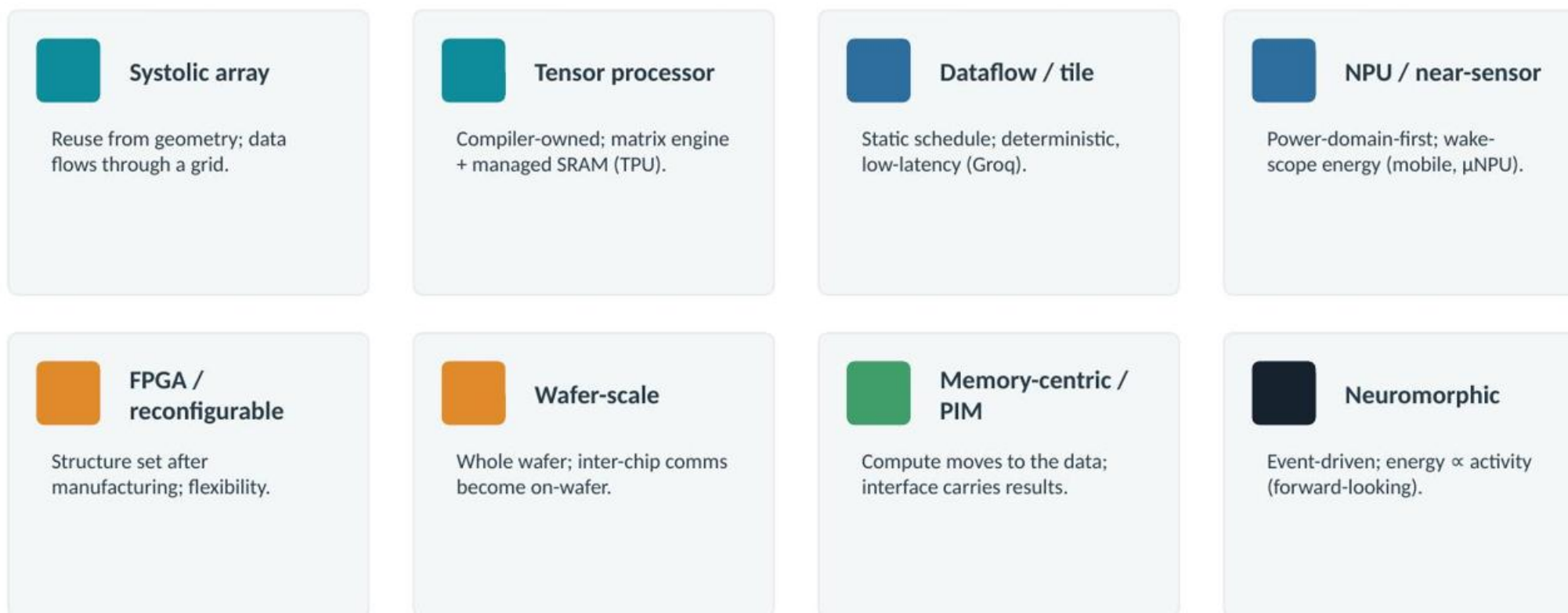
⇒ Every family is a displacement from the CPU/GPU baseline on these two axes — no point dominates.

TALKING POINTS

15 / 23

Two axes organize every family. Horizontal: WHEN execution uncertainty is resolved — runtime (GPU) → compile time (tensor/dataflow) → design time (fixed-function) → input-driven (neuromorphic). Vertical: WHERE state lives relative to arithmetic — DRAM → on-chip SRAM → inside memory. Every family is a displacement from the CPU/GPU baseline. No point dominates — each purchase of efficiency narrows an envelope — which is why the landscape stays permanently plural.

Eight families = eight contract changes



⇒ Grouped by contract changed, not by company — the taxonomy outlives the products.

TALKING POINTS

16 / 23

The eight families — each is ONE contract change, not "a faster chip." Read them as answers to "which promise did you renegotiate?": systolic = reuse geometry; tensor processor = compiler owns the schedule; dataflow = static, deterministic timing; NPU = design around power domains; FPGA = set structure after manufacturing; wafer-scale = make inter-chip comms on-wafer; PIM = move compute into memory; neuromorphic = let the input schedule the machine. Grouped by contract, not vendor — that's why the taxonomy outlives the product names.

NPU are the client's standard low-power AI block



- Universal in client SoCs (≥ 40 TOPS floor).
- Built for sustained / ambient work — not peak.
- Heavy generative inference still runs on the GPU.
- Coverage, not TOPS, decides real value.

⇒ **Structural claim:** wake-scope energy + cost floors sit outside the GPU contract, so the edge is where specialization is safest — not that NPUs win the client outright.

TALKING POINTS

17 / 23

IMPORTANT — this is an OBSERVATION about ubiquity + structural fit, NOT a claim that NPUs do most client AI compute. Hard facts (class A): every mainstream client SoC ships an NPU; Microsoft Copilot+ mandates ≥ 40 TOPS (Intel ~ 48 , AMD ~ 50 , Qualcomm 45-85, Apple Neural Engine in every chip since 2017). They're positioned for sustained/ambient low-power work. COUNTER-FACT: heavy local generative inference still runs mostly on the client GPU (CUDA/DirectML) — PCWorld's "the great NPU failure" — because NPU software/operator coverage lags; measured NPU utilization is low. So TOPS $\neq$ usage. The defensible claim is the takeaway: wake-scope energy and per-unit cost floors are structurally OUTSIDE the GPU contract, so the edge is where specialization is safest. If asked "observation or prediction?": mostly observation (ubiquity + structural niche) plus a conditional forward claim; the word "own" overstates and I've softened it.

Memory-centric designs attack the measured bounds

Compute moves to the data – three flavors



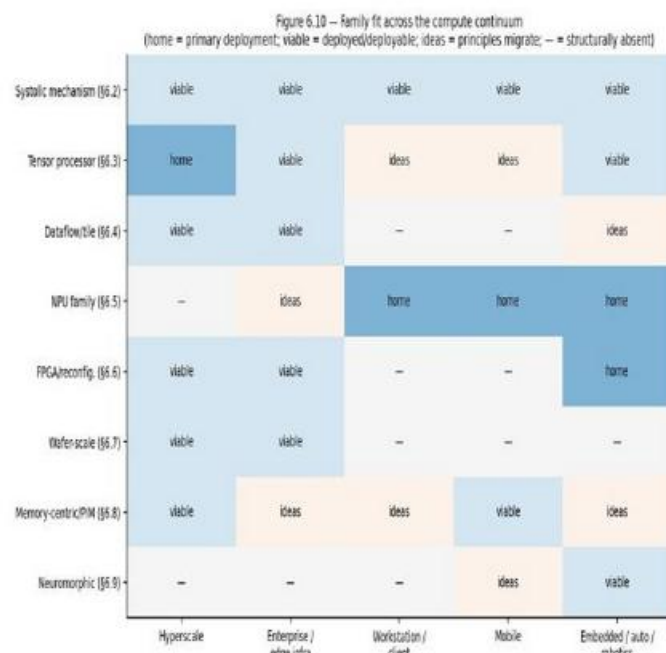
⇒ Best condition-3 case in the survey — gated only by the lack of a portable programming model.

TALKING POINTS

18 / 23

Processing-in-memory attacks exactly the bounds we measured (decode streaming, embedding gather, mobile wake energy). Three flavors: in-DRAM SIMD (Samsung Aquabolt-XL, UPMEM), all-SRAM inference (IBM NorthPole), tiered capacity (SambaNova SN40L). It has the STRONGEST condition-3 case in the survey — the lever genuinely changes the bound, doesn't just relocate it. Gated by ONE thing: no portable programming model. Watch for the first mainstream part whose datasheet advertises in-memory ops.

Same families, different binding constraints



⇒ Ideas migrate both ways; implementations diverge because each deployment prices the stressors differently.

TALKING POINTS

19 / 23

The same families appear from microwatt silicon to megawatt pods — resized against different binding constraints. Ideas migrate BOTH ways: unified memory and power discipline go up from phones into the datacenter; quantization and compilation come down. Implementations diverge because each deployment class prices the stressors differently (rack power vs battery, tenancy vs wake scope). Argues for ONE shared taxonomy, not two industries.

Three groups, one deciding variable

Grouped by how condition 4 resolves — not a product catalog



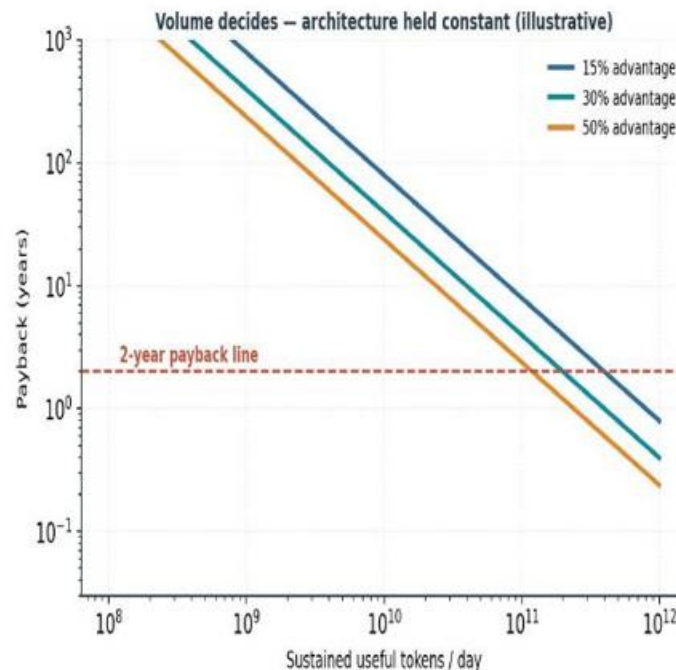
⇒ None of the challengers failed on architecture — every trajectory was decided by condition 4.

TALKING POINTS

20 / 23

The landscape in three buckets, read through condition 4 — NOT a product catalog. (1) GPU incumbents (NVIDIA, AMD): absorb bounds, sell rack-scale. (2) Hyperscaler tensor processors (TPU, Trainium, Maia, MTIA): condition 4 met at owner scale — every major cloud now builds one. (3) Independent challengers: Groq licensed into NVIDIA, Cerebras survives on anchor tenancy + IPO, Tenstorrent in acquisition talks, SambaNova an appliance vendor. Punchline: NONE failed on architecture — every outcome was decided by software capital, volume, and distribution.

Volume decides — architecture held constant



- Fixed cost $\approx$ software enablement.
- Recurring savings scale with volume.
- Flexibility has option value.

⇒ Below a volume threshold, no efficiency percentage rescues the case; above it, small percentages justify large programs.

TALKING POINTS

21 / 23

Economics in one chart (numbers illustrative). The fixed cost of specialization is dominated by SOFTWARE enablement (compiler, validation, migration), not silicon. Recurring savings scale with volume; the fixed cost doesn't. So there's a volume threshold: below it, no efficiency percentage rescues the case; above it, even small percentages justify huge programs. This is the arithmetic behind "same architecture, opposite verdicts" — hyperscale vs enterprise differ mainly in the denominator. Also price flexibility as option value: a 20% win today assumes the workload holds still.

What holds across the whole picture

A

Software converges on physics

Both transitions are software succeeding until it exposes a hard, measurable bound.

B

Necessity is decidable

A four-condition test returns specialize / market-no / stay-on-GPU — from measurable inputs.

C

Condition 4 dominates

Software capital, volume, and distribution decide outcomes more than silicon merit.

D

Families, not a successor

Each changes a contract the GPU cannot adopt without ceasing to be a GPU.

E

Bounds moved to memory & power

Capacity, bandwidth, fabric, and facility power now separate contenders — not FLOPS.

F

One continuum

The same taxonomy reads microwatt silicon and megawatt pods.

TALKING POINTS

22 / 23

Six findings, each traceable to evidence shown earlier. The two to leave ringing: (C) condition 4 — software capital — decides most outcomes, so audit the software stack before comparing silicon; and (E) the bounds have moved to memory, fabric, and facility power — arithmetic peak no longer separates serious contenders. If they remember one test, it's the four conditions.

THE ARGUMENT IN ONE SENTENCE

Software optimization succeeds until it exposes a bound; the bound is absorbed by the GPU platform, escaped by changing a contract, or lived with — and the deployment class decides which.

Everything else — signatures, contracts, families, economics — is the machinery that makes that choice decidable.

TALKING POINTS

23 / 23

Land the one-sentence summary and stop. Software succeeds until it exposes a bound; that bound is absorbed / escaped by changing a contract / lived with — and the DEPLOYMENT CLASS decides which. Invite the necessity test as the takeaway tool. Q&A prompts likely: NPU-vs-GPU on client, whether hyperscaler silicon kills NVIDIA (no — absorption + condition 4), and PIM timing.